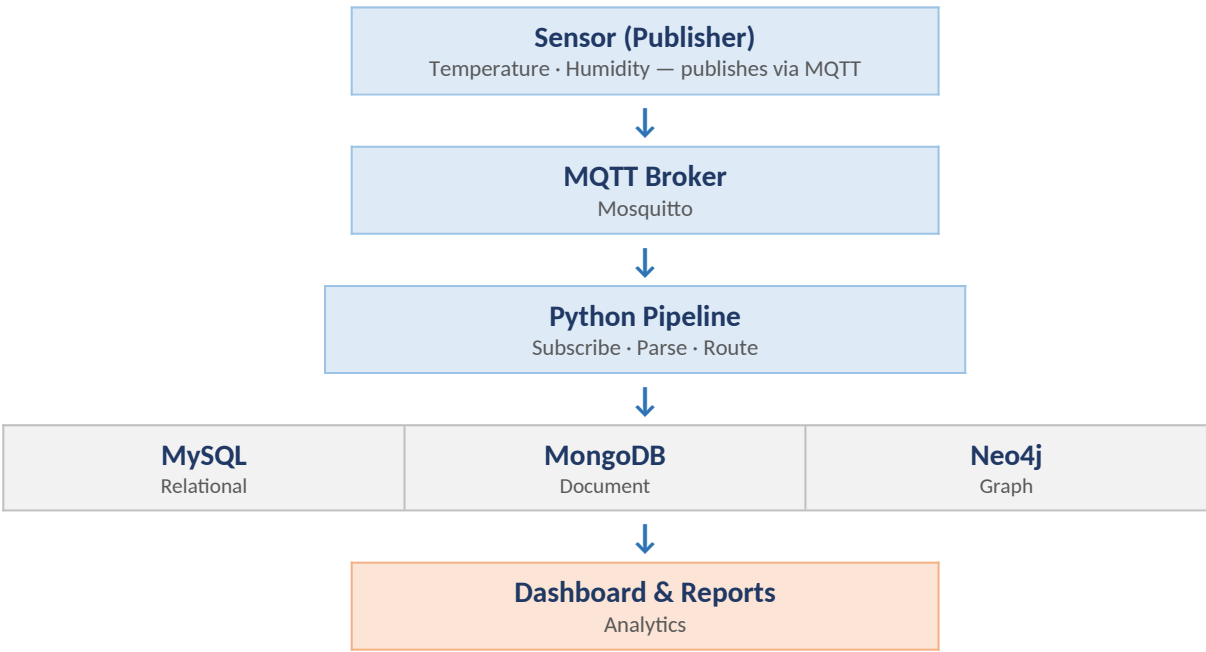




PROJECT REPORT

IoT-Based Environmental Monitoring System

From Physical Sensors to Digital Dashboards and Multi-Database Data Pipelines



STUDENT
Adonay Samson Gebrewold

MATRICOLA
570352

ACADEMIC YEAR
2025 – 2026

PROFESSOR
Prof. Armando Ruggeri

GITHUB
Donnie2444

PROJECT
DB-B1

Contents

1	Introduction	3
2	Project Motivation	3
3	System Architecture	3
4	Technologies Used	4
4.1	Python	4
4.2	MQTT and Mosquitto	4
4.3	MySQL	5
4.4	MongoDB	5
4.5	Neo4j	5
4.6	Docker	5
5	MQTT Communication Design	5
6	Data Processing Logic	6
7	Database Design	7
7.1	MySQL Relational Model	7
7.2	Relational Schema	7
7.3	ER Design Explanation	8
7.4	Normalization	8
7.5	MongoDB Document Model	9
7.6	Neo4j Graph Model	10
7.7	Why Three Database Models Were Used	10
8	Implementation Details	10
8.1	Docker Compose Configuration	11
8.2	MQTT Publisher Implementation	11
8.3	MQTT Subscriber Implementation	12
8.4	Environment Configuration	12
8.5	Python Dependencies	12
8.6	Execution Flow	12
9	Testing and Evaluation	13
9.1	Docker Services Test	13
9.2	MQTT Publisher Test	13
9.3	MQTT Subscriber Test	14
9.4	MySQL Storage and Analysis Test	14
9.5	MongoDB Storage and Aggregation Test	15

9.6	Neo4j Graph Test	16
9.7	Testing Summary	17
10	Results Discussion	17
11	Challenges Faced	17
12	Conclusion	18
13	Future Improvements	18
14	References	19

1 Introduction

The purpose of this project is to design and implement a database-oriented Internet of Things (IoT) monitoring system. The selected project topic is environmental monitoring. In this system, simulated IoT sensors collect data about temperature, humidity, and air quality. This type of system can be useful in rooms, laboratories, offices, or university buildings where environmental conditions need to be observed continuously.

The project uses Python to simulate environmental sensors and to process incoming sensor data. Communication between the sensor simulator and the data-processing application is handled through the MQTT protocol. MQTT is suitable for this kind of project because it is commonly used in IoT systems where small messages are exchanged between devices and servers.

The collected data is stored in three different database systems: MySQL, MongoDB, and Neo4j. Each database is used for a different purpose. MySQL stores the structured relational data. MongoDB stores the received MQTT messages as flexible JSON documents. Neo4j stores the relationships between sensors, locations, readings, and alerts as a graph.

The final system demonstrates how IoT data can be generated, transmitted, processed, stored, and analyzed using different database models.

2 Project Motivation

Environmental data is important in many real-world situations. For example, high temperature may damage electronic equipment, high humidity may affect comfort or laboratory conditions, and poor air quality may indicate an unhealthy environment. In a real system, physical IoT sensors would collect this information and send it to a server for processing.

Since physical sensors were not available for this project, the sensor data was simulated using Python. This approach still allows the main goal of the project to be achieved because the focus is not on the physical sensor hardware, but on the data flow, database storage, and processing logic.

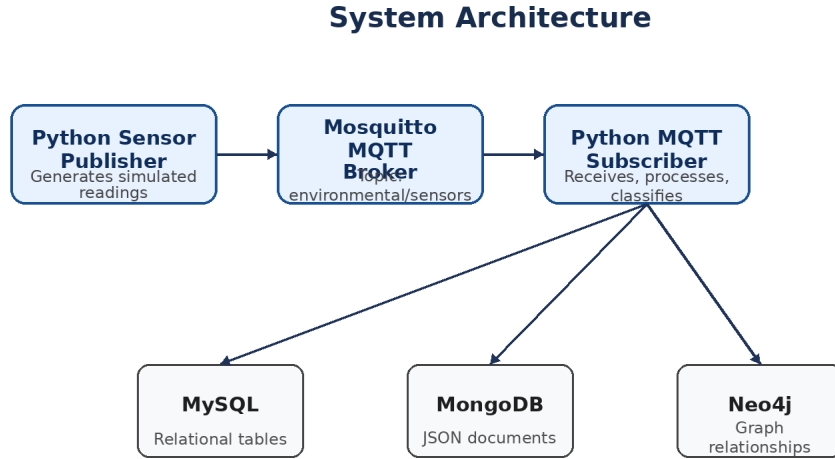
The main motivation of this project is to show how a DBMS-based system can manage continuous IoT data. The project also compares different database models by storing the same type of sensor information in relational, document, and graph form.

3 System Architecture

The system is based on a simple IoT data pipeline. A Python publisher program acts as the environmental sensor. It generates random values for temperature, humidity, and air quality. These values are converted into JSON messages and published to an MQTT

topic.

The Mosquitto MQTT broker receives the messages and delivers them to the Python subscriber. The subscriber receives each message, classifies the environmental condition, and stores the result in the databases.



Docker Compose runs the MQTT broker and database services; Python programs run from VS Code.

Figure 1: General architecture of the environmental monitoring system.

The general data flow is:

```

Python Sensor Publisher
  -> Mosquitto MQTT Broker
  -> Python MQTT Subscriber
  -> MySQL + MongoDB + Neo4j
  
```

Docker containers are used to run the required services. The project uses containers for Mosquitto, MySQL, MongoDB, and Neo4j. This makes the project easier to run because the services do not need to be installed manually on the operating system.

4 Technologies Used

4.1 Python

Python is used to create both the publisher and the subscriber. The publisher generates simulated sensor readings, while the subscriber receives the MQTT messages, processes the data, and inserts the results into the databases.

4.2 MQTT and Mosquitto

MQTT is used as the communication protocol between the sensor simulator and the processing application. Mosquitto is used as the MQTT broker. The broker receives

published messages and forwards them to subscribed clients.

4.3 MySQL

MySQL is used as the relational database. It stores structured data about locations, sensors, readings, and alerts. This database is useful for querying data using SQL.

4.4 MongoDB

MongoDB is used as a document database. It stores the received sensor messages in JSON-like document format. This is useful because MQTT messages are already structured as JSON data.

4.5 Neo4j

Neo4j is used as a graph database. It stores relationships such as sensor-to-location, sensor-to-reading, and reading-to-alert. This makes it easier to visualize and query connections between entities.

4.6 Docker

Docker is used to run the services in isolated containers. Docker Compose starts all required services with one command.

5 MQTT Communication Design

The MQTT communication has two main parts: the publisher and the subscriber.

The publisher represents the IoT sensor device. It randomly selects one of three sensors:

- S001 in Room A
- S002 in Room B
- S003 in Laboratory

For each reading, the publisher generates the sensor ID, location, temperature, humidity, air quality, and timestamp. The data is converted into JSON format and published to the MQTT topic:

```
environmental/sensors
```

An example message is:

```
{
  "sensor_id": "S003",
  "location": "Laboratory",
  "temperature": 31.8,
  "humidity": 72.96,
  "air_quality": 39,
  "timestamp": "2026-06-16 17:14:41"
}
```

The subscriber listens to the same MQTT topic. Whenever a message arrives, it decodes the JSON data, classifies the reading, and stores it in MySQL, MongoDB, and Neo4j.

6 Data Processing Logic

After receiving a sensor message, the subscriber classifies the reading into one of three statuses:

- NORMAL
- WARNING
- DANGER

A reading is classified as **DANGER** when at least one of the following conditions is true:

- Temperature is greater than 35 degrees Celsius.
- Humidity is greater than 80 percent.
- Air quality is greater than 150.

A reading is classified as **WARNING** when at least one of the following conditions is true:

- Temperature is greater than 30 degrees Celsius.
- Humidity is greater than 70 percent.
- Air quality is greater than 100.

If none of these conditions are true, the reading is classified as **NORMAL**.

This processing step is important because the system is not only storing raw data. It also adds meaning to the data before storage. When a dangerous or warning condition is detected, an alert is also created.

7 Database Design

7.1 MySQL Relational Model

MySQL is used to store structured relational data. The relational database contains four main tables: locations, sensors, readings, and alerts.

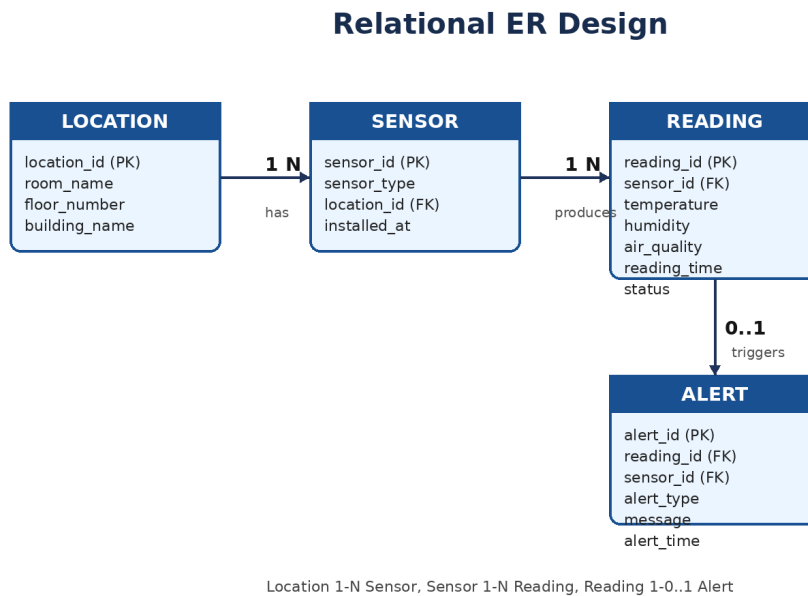


Figure 2: ER diagram of the relational database design.

7.2 Relational Schema

The relational schema used in MySQL is shown below.

```

locations(location_id PK, room_name UNIQUE, floor_number, building_name)

sensors(sensor_id PK, sensor_type, location_id FK, installed_at)

readings(reading_id PK, sensor_id FK, temperature, humidity,
          air_quality, reading_time, status, created_at)

alerts(alert_id PK, reading_id FK, sensor_id FK,
        alert_type, message, alert_time)
  
```

The foreign-key constraints are:

- sensors.location_id references locations.location_id
- readings.sensor_id references sensors.sensor_id
- alerts.reading_id references readings.reading_id

- `alerts.sensor_id` references `sensors.sensor_id`

These constraints prevent invalid data. For example, the system cannot insert a reading for a sensor that does not exist. Also, an alert must be linked to an existing reading and an existing sensor.

7.3 ER Design Explanation

The relational part of the system was designed using four main entities: Location, Sensor, Reading, and Alert. These entities represent the most important parts of the environmental monitoring system.

A Location represents a physical place where environmental data is collected, such as Room A, Room B, or the Laboratory. A Sensor is installed in one location and produces environmental readings. A Reading stores the actual values collected from a sensor, including temperature, humidity, air quality, timestamp, and status. An Alert is created when a reading contains abnormal or dangerous environmental values.

The main relationships are:

- One Location can have many Sensors.
- One Sensor belongs to one Location.
- One Sensor can produce many Readings.
- One Reading belongs to one Sensor.
- One Reading may generate one Alert if the values are warning or dangerous.
- One Alert is linked to one specific Reading.

In short form, the ER model can be described as:

```
Location 1 ---- N Sensor
Sensor   1 ---- N Reading
Reading  1 ---- 0..1 Alert
```

This design separates static information from dynamic information. Locations and sensors do not change often, while readings are continuously inserted as new MQTT messages arrive.

7.4 Normalization

The relational database was designed to avoid unnecessary repetition and to keep the data organized.

First Normal Form

The database satisfies First Normal Form because all table attributes contain atomic values. For example, temperature, humidity, and air quality are stored as separate columns. The system does not store multiple readings in one field.

Second Normal Form

The database satisfies Second Normal Form because every non-key attribute depends on the whole primary key. Most tables use a single-column primary key, such as `location_id`, `reading_id`, and `alert_id`. Therefore, there are no partial dependencies.

Third Normal Form

The database satisfies Third Normal Form because non-key attributes do not depend on other non-key attributes. For example, sensor location information is not stored repeatedly inside the readings table. Instead, the readings table stores `sensor_id`, and the sensor is connected to its location through the sensors table.

BCNF

The schema also follows the idea of Boyce-Codd Normal Form because the main determinants in the tables are candidate keys. In the locations table, `location_id` identifies the location, and `room_name` is also unique. In the sensors table, `sensor_id` identifies each sensor. In the readings table, `reading_id` identifies each reading. In the alerts table, `alert_id` identifies each alert.

7.5 MongoDB Document Model

MongoDB is used to store the MQTT messages in document format. Each document represents one sensor reading and includes both the original data and the processed result.

A MongoDB document contains the MySQL reading ID, sensor ID, location, temperature, humidity, air quality, timestamp, status, alert information, and storage time.

An example document structure is:

```
{
  "mysql_reading_id": 1,
  "sensor_id": "S001",
  "location": "Room A",
  "temperature": 28.21,
  "humidity": 63.68,
  "air_quality": 121,
  "timestamp": "2026-06-16 17:14:51",
  "status": "WARNING",
  "alert": {
```

```
"alert_type": "ENVIRONMENT_WARNING",  
  "message": "One or more environmental values are above the normal range."  
},  
"stored_at": "2026-06-16 17:14:51"  
}
```

MongoDB is useful in this project because the data received from MQTT is already in JSON format. This makes it natural to store the messages as documents.

7.6 Neo4j Graph Model

Neo4j is used to represent the data as a graph. This database is useful when relationships between entities are important.

The graph model contains the following nodes:

- Sensor
- Location
- Reading
- Alert

The main relationships are:

```
Sensor -[:LOCATED_IN]-> Location  
Sensor -[:PRODUCED]-> Reading  
Reading -[:TRIGGERED]-> Alert
```

This model makes it easy to visualize how sensors, locations, readings, and alerts are connected.

7.7 Why Three Database Models Were Used

The project uses MySQL, MongoDB, and Neo4j because each database model has a different strength. MySQL is suitable for structured data, tables, primary keys, foreign keys, and SQL queries. MongoDB is suitable for storing MQTT messages in JSON document format. Neo4j is useful for representing relationships as a graph.

Using these three databases shows that the same IoT data can be represented in different ways depending on the purpose of the database.

8 Implementation Details

The implementation of the project is divided into four main parts: Docker service configuration, MQTT publisher, MQTT subscriber, and database storage. This separation makes the project easier to understand, test, and maintain.

The main project structure is:

```
environmental_monitoring_mqtt/  
|-- docker-compose.yml  
|-- mosquitto/config/mosquitto.conf  
|-- database/mysql_schema.sql  
|-- publisher/sensor_publisher.py  
|-- subscriber/mqtt_subscriber.py  
|-- requirements.txt  
|-- .env.example  
'-- screenshots/
```

8.1 Docker Compose Configuration

Docker Compose is used to start all required services for the project. The file `docker-compose.yml` defines four containers:

- `env_mqtt_broker`
- `env_mysql_db`
- `env_mongodb_db`
- `env_neo4j_db`

The Mosquitto container provides the MQTT broker and listens on port 1883. The MySQL container creates the `env_monitoring` database. The MongoDB container stores the MQTT sensor messages as documents. The Neo4j container provides both the browser interface on port 7474 and the Bolt connection on port 7687.

The services were started using:

```
docker compose up -d
```

The command `docker ps` was then used to confirm that all containers were running correctly.

8.2 MQTT Publisher Implementation

The file `sensor_publisher.py` is responsible for simulating IoT environmental sensors. It works as the data producer in the system.

The publisher connects to the MQTT broker using `localhost` and port 1883. It simulates three sensors: S001, S002, and S003. Each generated message contains the sensor ID, location, temperature, humidity, air quality, and timestamp.

The publisher sends one message every two seconds. This confirms that the publisher behaves like an IoT device sending live sensor data.

8.3 MQTT Subscriber Implementation

The file `mqtt_subscriber.py` is responsible for receiving MQTT messages, processing the sensor data, and storing the result in the databases.

When a new message arrives, the subscriber decodes the JSON message and passes it to the processing function. The processing function classifies the reading as NORMAL, WARNING, or DANGER, then inserts the processed data into MySQL, MongoDB, and Neo4j.

8.4 Environment Configuration

The project uses a `.env` file to store configuration values. This is better than writing connection details directly inside the Python code.

The `.env` file contains MQTT connection settings, MySQL settings, MongoDB settings, and Neo4j settings. Using environment variables makes the project easier to change.

8.5 Python Dependencies

The required Python libraries are listed in `requirements.txt`. The main libraries are:

- paho-mqtt
- mysql-connector-python
- pymongo
- neo4j
- python-dotenv

The dependencies were installed using:

```
pip install -r requirements.txt
```

8.6 Execution Flow

The system was executed in the following order:

```
docker compose up -d
docker ps
python subscriber/mqtt_subscriber.py
python publisher/sensor_publisher.py
```

This order is important. The subscriber should run before the publisher so that it is already listening when the first MQTT messages are published.

Project Setup and Python Environment

The following screenshots are placed after the implementation explanation because they show how the project was prepared before execution. They verify the working directory, Python virtual environment, and dependency installation.

```
PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter> dir

Directory: C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter

Mode                LastWriteTime         Length Name
----                -
d-----          18/06/2026   22:46             .venv
d-----          21/06/2026   23:17     environmental_monitoring_mqtt

PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter> cd environmental_monitoring_mqtt
```

Figure E1: Project directory opened from PowerShell before entering the main `environmental_monitoring_mqtt` folder.

```
PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt>
```

Figure E2: Activation of the Python virtual environment using `.\.venv\Scripts\Activate.ps1`.

```
(.venv) PS C:\Users\adona.LAPTOP-FTQ3C4HH\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt> pip install -r requirements.txt
Requirement already satisfied: paho-mqtt==1.6.1 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from -r requirements.txt (line 1)) (1.6.1)
Requirement already satisfied: mysql-connector-python==9.1.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from -r requirements.txt (line 2)) (9.1.0)
Requirement already satisfied: pymongo==4.10.1 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from -r requirements.txt (line 3)) (4.10.1)
Requirement already satisfied: neo4j==5.27.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from -r requirements.txt (line 4)) (5.27.0)
Requirement already satisfied: python-dotenv==1.0.1 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from -r requirements.txt (line 5)) (1.0.1)
Requirement already satisfied: dnspython<3.0.0,>=1.16.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from pymongo==4.10.1->r requirements.txt (line 3)) (2.8.0)
Requirement already satisfied: pytz in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\.venv\lib\site-packages (from neo4j==5.27.0->-r requirements.txt (line 4)) (2026.2)

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Figure E3: Installation and verification of the main Python dependencies from `requirements.txt`.

Dashboard Dependency Installation

The Streamlit dashboard uses a separate requirements file. This screenshot is inserted in the dependency section because it verifies the correct dashboard installation command and the packages needed for the dashboard interface.

```
ackages (from streamlit->r requirements_dashboard.txt (line 1)) (0.49.0)
Requirement already satisfied: httptools<=0.6.3 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (0.8.0)
Requirement already satisfied: anyio<=4.0.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (4.14.0)
Requirement already satisfied: python-multipart<=0.0.10 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (0.0.32)
Requirement already satisfied: websockets<=12.0.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (16.0)
Requirement already satisfied: itsdangerous<=2.1.2 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from streamlit->r requirements_dashboard.txt (line 1)) (6.0.0)
Requirement already satisfied: python-dateutil<=2.8.2 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from pandas->r requirements_dashboard.txt (line 2)) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from pandas->r requirements_dashboard.txt (line 2)) (2026.2)
Requirement already satisfied: Jinja2 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from altair<=5.4.0,!>5.4.1,<7,>=4.0->streamlit->r requirements_dashboard.txt (line 1)) (3.1.6)
Requirement already satisfied: jsonschema<=3.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from altair<=5.4.0,!>5.4.1,<7,>=4.0->streamlit->r requirements_dashboard.txt (line 1)) (4.26.0)
Requirement already satisfied: narwhals<=2.4.0 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from altair<=5.4.0,!>5.4.1,<7,>=4.0->streamlit->r requirements_dashboard.txt (line 1)) (2.22.1)
Requirement already satisfied: colorama in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from click<9,>=7.0->streamlit->r requirements_dashboard.txt (line 1)) (0.4.6)
Requirement already satisfied: gitdb<5,>=4.0.1 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from gitpython<=3.1.19,<4,>=3.0.7->streamlit->r requirements_dashboard.txt (line 1)) (4.0.12)
Requirement already satisfied: smmap<6,>=3.0.1 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from gitdb<5,>=4.0.1->gitpython<=3.1.19,<4,>=3.0.7->streamlit->r requirements_dashboard.txt (line 1)) (5.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from requests<3,>=2.27->streamlit->r requirements_dashboard.txt (line 1)) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from requests<3,>=2.27->streamlit->r requirements_dashboard.txt (line 1)) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in c:\users\adona.laptop-ftq3c4hh\downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt\venv\lib\site-packages (from requests<3,>=2.27->streamlit->r requirements_dashboard.txt (line 1)) (2.7.0)
```

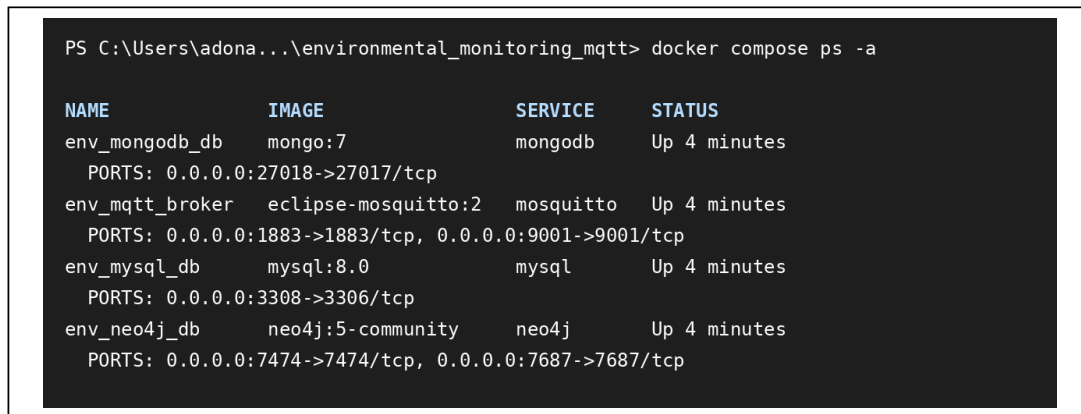
Figure E4: Correct dashboard dependency installation using `pip install -r requirements_dashboard.txt`.

9 Testing and Evaluation

After implementing the system, several tests were carried out to make sure that each part of the project worked correctly. The testing focused on checking that the Docker services were running, confirming that MQTT messages were being published and received, verifying that the data was stored in the databases, and running analysis queries on the stored data.

9.1 Docker Services Test

The command `docker ps` was used to check whether all required services were running correctly. The output showed that the MQTT broker, MySQL database, MongoDB database, and Neo4j database were all active.



```
PS C:\Users\adona...\environmental_monitoring_mqtt> docker compose ps -a
```

NAME	IMAGE	SERVICE	STATUS
env_mongodb_db	mongo:7	mongodb	Up 4 minutes
PORTS: 0.0.0.0:27018->27017/tcp			
env_mqtt_broker	eclipse-mosquitto:2	mosquitto	Up 4 minutes
PORTS: 0.0.0.0:1883->1883/tcp, 0.0.0.0:9001->9001/tcp			
env_mysql_db	mysql:8.0	mysql	Up 4 minutes
PORTS: 0.0.0.0:3308->3306/tcp			
env_neo4j_db	neo4j:5-community	neo4j	Up 4 minutes
PORTS: 0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp			

Figure 3: Docker containers running successfully

9.2 MQTT Publisher Test

The publisher was tested by running:

```
python publisher/sensor_publisher.py
```

The terminal showed JSON messages being published to the MQTT topic `environmental/sensors`.

Docker Environment Verification

Docker Compose was used to run the services required by the project. The status output verifies that MongoDB, Mosquitto, MySQL, and Neo4j were running as containers before executing the Python programs.

```
PS C:\Users\adona...\environmental_monitoring_mqtt> docker compose ps -a
```

NAME	IMAGE	SERVICE	STATUS
env_mongodb_db	mongo:7	mongodb	Up 4 minutes
PORTS: 0.0.0.0:27018->27017/tcp			
env_mqtt_broker	eclipse-mosquitto:2	mosquitto	Up 4 minutes
PORTS: 0.0.0.0:1883->1883/tcp, 0.0.0.0:9001->9001/tcp			
env_mysql_db	mysql:8.0	mysql	Up 4 minutes
PORTS: 0.0.0.0:3308->3306/tcp			
env_neo4j_db	neo4j:5-community	neo4j	Up 4 minutes
PORTS: 0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp			

Figure E5: Docker Compose status showing the running MongoDB, MQTT broker, MySQL, and Neo4j containers.

```
PS C:\Users\adona...\environmental_monitoring_mqtt> docker compose ps -a
```

NAME	IMAGE	SERVICE	STATUS
env_mongodb_db	mongo:7	mongodb	Up 4 minutes
PORTS: 0.0.0.0:27018->27017/tcp			
env_mqtt_broker	eclipse-mosquitto:2	mosquitto	Up 4 minutes
PORTS: 0.0.0.0:1883->1883/tcp, 0.0.0.0:9001->9001/tcp			
env_mysql_db	mysql:8.0	mysql	Up 4 minutes
PORTS: 0.0.0.0:3308->3306/tcp			
env_neo4j_db	neo4j:5-community	neo4j	Up 4 minutes
PORTS: 0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp			

Figure E6: Clean terminal reproduction of the same Docker Compose service status.

MQTT Publisher and Subscriber Execution

The subscriber was started first so that it could listen on the environmental/sensors topic. The publisher then sent JSON readings to the MQTT broker, and the subscriber received, classified, and stored the data.

```

C:\Users\adona.LAPTOP-FQ3C4H\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt>python subscriber\mqtt_subscriber.py
Subscriber is running. Press CTRL+C to stop.
Connected to MQTT broker at localhost:1883
Subscribed to topic: environmental/sensors

```

Figure E7: MQTT subscriber connected to the broker and subscribed to environmental/sensors.

```

(.venv) PS C:\Users\adona.LAPTOP-FQ3C4H\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt>python publisher\sensor_publisher.py
Connected to MQTT broker at localhost:1883
Publishing sensor data to topic: environmental/sensors
Press CTRL+C to stop.

Published: {"sensor_id": "S002", "location": "Room B", "temperature": 30.24, "humidity": 66.13, "air_quality": 139, "timestamp": "2026-06-21 23:44:20"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 33.51, "humidity": 80.86, "air_quality": 132, "timestamp": "2026-06-21 23:44:22"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 28.6, "humidity": 52.93, "air_quality": 57, "timestamp": "2026-06-21 23:44:24"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 20.91, "humidity": 69.29, "air_quality": 156, "timestamp": "2026-06-21 23:44:26"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 27.84, "humidity": 50.11, "air_quality": 100, "timestamp": "2026-06-21 23:44:28"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 35.3, "humidity": 52.9, "air_quality": 130, "timestamp": "2026-06-21 23:44:30"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 23.59, "humidity": 54.34, "air_quality": 36, "timestamp": "2026-06-21 23:44:32"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 21.51, "humidity": 63.86, "air_quality": 161, "timestamp": "2026-06-21 23:44:34"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 29.6, "humidity": 66.97, "air_quality": 168, "timestamp": "2026-06-21 23:44:36"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 18.24, "humidity": 68.74, "air_quality": 96, "timestamp": "2026-06-21 23:44:38"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 23.77, "humidity": 77.07, "air_quality": 54, "timestamp": "2026-06-21 23:44:40"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 28.34, "humidity": 56.02, "air_quality": 162, "timestamp": "2026-06-21 23:44:42"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 19.86, "humidity": 50.76, "air_quality": 43, "timestamp": "2026-06-21 23:44:44"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 30.07, "humidity": 65.55, "air_quality": 111, "timestamp": "2026-06-21 23:44:46"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 36.44, "humidity": 72.76, "air_quality": 43, "timestamp": "2026-06-21 23:44:48"}

```

Figure E8: MQTT publisher sending simulated JSON readings to the environmental/sensors topic.

```

(.venv) PS C:\Users\adona.LAPTOP-FQ3C4H\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt>python subscriber\mqtt_subscriber.py
Subscriber is running. Press CTRL+C to stop.
Connected to MQTT broker at localhost:1883
Subscribed to topic: environmental/sensors

Stored reading #1 | Sensor=S002 | Status=WARNING | Temp=30.24 | Humidity=66.13 | AQ=139
Stored reading #2 | Sensor=S002 | Status=DANGER | Temp=33.51 | Humidity=80.86 | AQ=132
Stored reading #3 | Sensor=S003 | Status=NORMAL | Temp=28.6 | Humidity=52.93 | AQ=57
Stored reading #4 | Sensor=S003 | Status=DANGER | Temp=20.91 | Humidity=69.29 | AQ=156
Stored reading #5 | Sensor=S002 | Status=NORMAL | Temp=27.84 | Humidity=50.11 | AQ=100
Stored reading #6 | Sensor=S001 | Status=DANGER | Temp=35.3 | Humidity=52.9 | AQ=130
Stored reading #7 | Sensor=S002 | Status=NORMAL | Temp=23.59 | Humidity=54.34 | AQ=36
Stored reading #8 | Sensor=S003 | Status=DANGER | Temp=21.51 | Humidity=63.86 | AQ=161
Stored reading #9 | Sensor=S002 | Status=DANGER | Temp=29.6 | Humidity=66.97 | AQ=168
Stored reading #10 | Sensor=S003 | Status=NORMAL | Temp=18.24 | Humidity=68.74 | AQ=96
Stored reading #11 | Sensor=S001 | Status=WARNING | Temp=23.77 | Humidity=77.07 | AQ=54
Stored reading #12 | Sensor=S002 | Status=DANGER | Temp=28.34 | Humidity=56.02 | AQ=162
Stored reading #13 | Sensor=S001 | Status=NORMAL | Temp=19.86 | Humidity=50.76 | AQ=43
Stored reading #14 | Sensor=S003 | Status=WARNING | Temp=30.07 | Humidity=65.55 | AQ=111
Stored reading #15 | Sensor=S001 | Status=DANGER | Temp=36.44 | Humidity=72.76 | AQ=43
Stored reading #16 | Sensor=S003 | Status=WARNING | Temp=24.45 | Humidity=42.74 | AQ=95
Stored reading #17 | Sensor=S003 | Status=WARNING | Temp=34.56 | Humidity=44.92 | AQ=95
Stored reading #18 | Sensor=S003 | Status=WARNING | Temp=21.26 | Humidity=67.98 | AQ=145
Stored reading #19 | Sensor=S002 | Status=NORMAL | Temp=27.14 | Humidity=46.79 | AQ=52
Stored reading #20 | Sensor=S001 | Status=WARNING | Temp=30.27 | Humidity=56.45 | AQ=40
Stored reading #21 | Sensor=S002 | Status=DANGER | Temp=23.51 | Humidity=80.06 | AQ=130
Stored reading #22 | Sensor=S003 | Status=DANGER | Temp=37.67 | Humidity=49.89 | AQ=178
Stored reading #23 | Sensor=S002 | Status=WARNING | Temp=32.41 | Humidity=74.73 | AQ=40
Stored reading #24 | Sensor=S001 | Status=WARNING | Temp=21.8 | Humidity=40.96 | AQ=116
Stored reading #25 | Sensor=S001 | Status=DANGER | Temp=18.8 | Humidity=60.79 | AQ=155
Stored reading #26 | Sensor=S002 | Status=WARNING | Temp=31.41 | Humidity=71.46 | AQ=150
Stored reading #27 | Sensor=S001 | Status=DANGER | Temp=26.28 | Humidity=43.03 | AQ=179
Stored reading #28 | Sensor=S002 | Status=DANGER | Temp=29.7 | Humidity=47.57 | AQ=159

```

Figure E9: Subscriber receiving messages, assigning NORMAL/WARNING/DANGER status, and storing readings.

```
(.venv) PS C:\Users\adona.LAPTOP-FTQ3C4H1\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt> python publisher\sensor_publisher.py

Connected to MQTT broker at localhost:1883
Publishing sensor data to topic: environmental/sensors
Press CTRL+C to stop.

Published: {"sensor_id": "S002", "location": "Room B", "temperature": 30.24, "humidity": 66.13, "air_quality": 139, "timestamp": "2026-06-21 23:44:20"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 33.51, "humidity": 80.86, "air_quality": 132, "timestamp": "2026-06-21 23:44:22"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 28.6, "humidity": 52.93, "air_quality": 57, "timestamp": "2026-06-21 23:44:24"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 20.91, "humidity": 60.29, "air_quality": 156, "timestamp": "2026-06-21 23:44:26"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 27.84, "humidity": 50.11, "air_quality": 100, "timestamp": "2026-06-21 23:44:28"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 35.3, "humidity": 52.9, "air_quality": 130, "timestamp": "2026-06-21 23:44:30"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 23.59, "humidity": 54.34, "air_quality": 36, "timestamp": "2026-06-21 23:44:32"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 21.51, "humidity": 63.86, "air_quality": 161, "timestamp": "2026-06-21 23:44:34"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 29.6, "humidity": 66.97, "air_quality": 168, "timestamp": "2026-06-21 23:44:36"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 18.24, "humidity": 68.74, "air_quality": 96, "timestamp": "2026-06-21 23:44:38"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 23.77, "humidity": 77.07, "air_quality": 54, "timestamp": "2026-06-21 23:44:40"}
Published: {"sensor_id": "S002", "location": "Room B", "temperature": 28.34, "humidity": 56.02, "air_quality": 162, "timestamp": "2026-06-21 23:44:42"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 19.86, "humidity": 50.76, "air_quality": 43, "timestamp": "2026-06-21 23:44:44"}
Published: {"sensor_id": "S003", "location": "Laboratory", "temperature": 30.07, "humidity": 65.55, "air_quality": 111, "timestamp": "2026-06-21 23:44:46"}
Published: {"sensor_id": "S001", "location": "Room A", "temperature": 36.44, "humidity": 72.76, "air_quality": 43, "timestamp": "2026-06-21 23:44:48"}
```

Figure 4: MQTT publisher sending environmental sensor data

9.3 MQTT Subscriber Test

The subscriber was tested by running:

```
python subscriber/mqtt_subscriber.py
```

The subscriber connected to the MQTT broker and received messages from the topic `environmental/sensors`. It processed the messages and stored the readings in the databases.

```
(.venv) PS C:\Users\adona.LAPTOP-FTQ3C4H1\Downloads\environmental_monitoring_mqtt_starter\environmental_monitoring_mqtt> python subscriber\mqtt_subscriber.py

Subscriber is running. Press CTRL+C to stop.
Connected to MQTT broker at localhost:1883
Subscribed to topic: environmental/sensors

Stored reading #1 | Sensor-S002 | Status-WARNING | Temp-30.24 | Humidity-66.13 | AQ-139
Stored reading #2 | Sensor-S002 | Status-DANGER | Temp-33.51 | Humidity-80.86 | AQ-132
Stored reading #3 | Sensor-S003 | Status-NORMAL | Temp-28.6 | Humidity-52.93 | AQ-57
Stored reading #4 | Sensor-S003 | Status-DANGER | Temp-20.91 | Humidity-60.29 | AQ-156
Stored reading #5 | Sensor-S002 | Status-NORMAL | Temp-27.84 | Humidity-50.11 | AQ-100
Stored reading #6 | Sensor-S001 | Status-DANGER | Temp-35.3 | Humidity-52.9 | AQ-130
Stored reading #7 | Sensor-S002 | Status-NORMAL | Temp-23.59 | Humidity-54.34 | AQ-36
Stored reading #8 | Sensor-S003 | Status-DANGER | Temp-21.51 | Humidity-63.86 | AQ-161
Stored reading #9 | Sensor-S002 | Status-DANGER | Temp-29.6 | Humidity-66.97 | AQ-168
Stored reading #10 | Sensor-S003 | Status-NORMAL | Temp-18.24 | Humidity-68.74 | AQ-96
Stored reading #11 | Sensor-S001 | Status-WARNING | Temp-23.77 | Humidity-77.07 | AQ-54
Stored reading #12 | Sensor-S002 | Status-DANGER | Temp-28.34 | Humidity-56.02 | AQ-162
Stored reading #13 | Sensor-S001 | Status-NORMAL | Temp-19.86 | Humidity-50.76 | AQ-43
Stored reading #14 | Sensor-S003 | Status-WARNING | Temp-30.07 | Humidity-65.55 | AQ-111
Stored reading #15 | Sensor-S001 | Status-DANGER | Temp-36.44 | Humidity-72.76 | AQ-43
Stored reading #16 | Sensor-S003 | Status-WARNING | Temp-24.45 | Humidity-42.26 | AQ-142
Stored reading #17 | Sensor-S003 | Status-WARNING | Temp-34.56 | Humidity-44.92 | AQ-95
Stored reading #18 | Sensor-S003 | Status-WARNING | Temp-21.26 | Humidity-67.98 | AQ-145
Stored reading #19 | Sensor-S002 | Status-NORMAL | Temp-27.14 | Humidity-46.79 | AQ-92
Stored reading #20 | Sensor-S001 | Status-WARNING | Temp-30.27 | Humidity-56.45 | AQ-40
Stored reading #21 | Sensor-S002 | Status-DANGER | Temp-23.51 | Humidity-80.86 | AQ-130
Stored reading #22 | Sensor-S003 | Status-DANGER | Temp-37.67 | Humidity-49.89 | AQ-178
Stored reading #23 | Sensor-S002 | Status-WARNING | Temp-22.42 | Humidity-24.23 | AQ-48
Stored reading #24 | Sensor-S001 | Status-WARNING | Temp-21.8 | Humidity-40.96 | AQ-116
Stored reading #25 | Sensor-S001 | Status-DANGER | Temp-18.8 | Humidity-60.79 | AQ-155
Stored reading #26 | Sensor-S002 | Status-WARNING | Temp-21.43 | Humidity-71.46 | AQ-159
Stored reading #27 | Sensor-S001 | Status-DANGER | Temp-20.28 | Humidity-43.81 | AQ-179
Stored reading #28 | Sensor-S002 | Status-DANGER | Temp-29.7 | Humidity-47.57 | AQ-159
```

Figure 5: MQTT subscriber receiving and storing sensor readings

9.4 MySQL Storage and Analysis Test

MySQL was tested using SQL queries. The readings table was checked using:

```
SELECT * FROM readings ORDER BY reading_id DESC LIMIT 10;
```

The following queries were also used to analyze the stored readings:

```
SELECT COUNT(*) AS total_readings FROM readings;
```

```
SELECT sensor_id, COUNT(*) AS number_of_readings
FROM readings
```

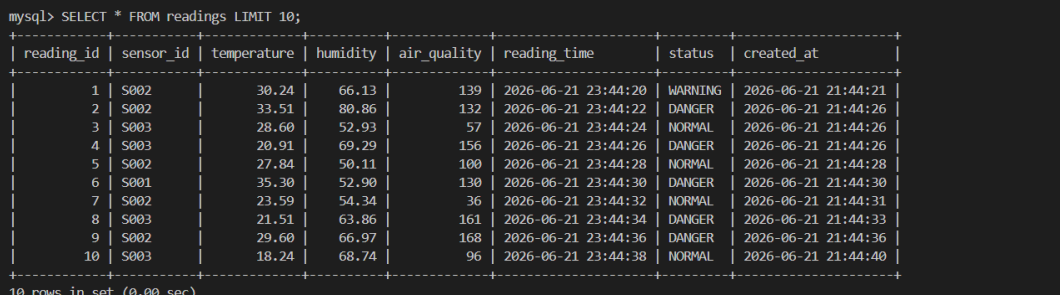
```

GROUP BY sensor_id;

SELECT status, COUNT(*) AS total
FROM readings
GROUP BY status;

SELECT
    sensor_id,
    ROUND(AVG(temperature), 2) AS avg_temperature,
    ROUND(AVG(humidity), 2) AS avg_humidity,
    ROUND(AVG(air_quality), 2) AS avg_air_quality
FROM readings
GROUP BY sensor_id;

```



```
mysql> SELECT * FROM readings LIMIT 10;
```

reading_id	sensor_id	temperature	humidity	air_quality	reading_time	status	created_at
1	S002	30.24	66.13	139	2026-06-21 23:44:20	WARNING	2026-06-21 21:44:21
2	S002	33.51	80.86	132	2026-06-21 23:44:22	DANGER	2026-06-21 21:44:26
3	S003	28.60	52.93	57	2026-06-21 23:44:24	NORMAL	2026-06-21 21:44:26
4	S003	20.91	69.29	156	2026-06-21 23:44:26	DANGER	2026-06-21 21:44:26
5	S002	27.84	50.11	100	2026-06-21 23:44:28	NORMAL	2026-06-21 21:44:28
6	S001	35.30	52.90	130	2026-06-21 23:44:30	DANGER	2026-06-21 21:44:30
7	S002	23.59	54.34	36	2026-06-21 23:44:32	NORMAL	2026-06-21 21:44:31
8	S003	21.51	63.86	161	2026-06-21 23:44:34	DANGER	2026-06-21 21:44:33
9	S002	29.60	66.97	168	2026-06-21 23:44:36	DANGER	2026-06-21 21:44:36
10	S003	18.24	68.74	96	2026-06-21 23:44:38	NORMAL	2026-06-21 21:44:40

10 rows in set (0.00 sec)

Figure 6: MySQL readings and analysis queries

9.5 MongoDB Storage and Aggregation Test

MongoDB was tested by checking the `sensor_messages` collection. The following commands were used:

```

use env_monitoring
db.sensor_messages.countDocuments()
db.sensor_messages.find().limit(5)

```

MongoDB was also tested using an aggregation query:

```

db.sensor_messages.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      total_messages: { $sum: 1 },
      avg_temperature: { $avg: "$temperature" },
      avg_humidity: { $avg: "$humidity" },
      avg_air_quality: { $avg: "$air_quality" }
    }
  }
])

```

MySQL Storage Verification

MySQL was checked from inside the Docker container. The screenshots prove that the expected relational tables exist and that the readings table contains sensor data generated by the MQTT pipeline.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_env_monitoring |
+-----+
| alerts                    |
| locations                 |
| readings                  |
| sensors                   |
+-----+
4 rows in set (0.01 sec)
```

Figure E10: MySQL tables created for locations, sensors, readings, and alerts.

```
mysql> SELECT COUNT(*) FROM readings;
+-----+
| COUNT(*) |
+-----+
|        168 |
+-----+
1 row in set (0.00 sec)
```

Figure E11: SQL count query showing stored readings in MySQL.

```
mysql> SELECT * FROM readings LIMIT 10;
+-----+-----+-----+-----+-----+-----+-----+-----+
| reading_id | sensor_id | temperature | humidity | air_quality | reading_time | status | created_at |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | S002 | 30.24 | 66.13 | 139 | 2026-06-21 23:44:20 | WARNING | 2026-06-21 21:44:21 |
| 2 | S002 | 33.51 | 80.86 | 132 | 2026-06-21 23:44:22 | DANGER | 2026-06-21 21:44:26 |
| 3 | S003 | 28.60 | 52.93 | 57 | 2026-06-21 23:44:24 | NORMAL | 2026-06-21 21:44:26 |
| 4 | S003 | 20.91 | 69.29 | 156 | 2026-06-21 23:44:26 | DANGER | 2026-06-21 21:44:26 |
| 5 | S002 | 27.84 | 50.11 | 100 | 2026-06-21 23:44:28 | NORMAL | 2026-06-21 21:44:28 |
| 6 | S001 | 35.30 | 52.90 | 130 | 2026-06-21 23:44:30 | DANGER | 2026-06-21 21:44:30 |
| 7 | S002 | 23.59 | 54.34 | 36 | 2026-06-21 23:44:32 | NORMAL | 2026-06-21 21:44:31 |
| 8 | S003 | 21.51 | 63.86 | 161 | 2026-06-21 23:44:34 | DANGER | 2026-06-21 21:44:33 |
| 9 | S002 | 29.60 | 66.97 | 168 | 2026-06-21 23:44:36 | DANGER | 2026-06-21 21:44:36 |
| 10 | S003 | 18.24 | 68.74 | 96 | 2026-06-21 23:44:38 | NORMAL | 2026-06-21 21:44:40 |
+-----+-----+-----+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

Figure E12: Sample records stored in the MySQL readings table.

MongoDB Document Storage Verification

MongoDB stores the received MQTT messages as document-style records. The output shows that the `sensor_messages` collection contains the sensor ID, location, environmental values, timestamp, status, and alert information.

```
env_monitoring> db.sensor_messages.find().pretty()
[
  {
    _id: ObjectId('6a385b3439cb69b7f001be5d'),
    mysql_reading_id: 1,
    sensor_id: 'S002',
    location: 'Room B',
    temperature: 30.24,
    humidity: 66.13,
    air_quality: 139,
    timestamp: '2026-06-21 23:44:20',
    status: 'WARNING',
    alert: { alert_type: 'ENVIRONMENT_WARNING', message: 'One or more environmental values are above the normal range.' },
    stored_at: '2026-06-21 23:44:20'
  },
  { mysql_reading_id: 2, sensor_id: 'S002', location: 'Room B', temperature: 33.51, humidity: 80.86, air_quality: 132, status: 'DANGER' }
]
```

Figure E13: MongoDB `sensor_messages` collection showing stored MQTT documents.

Raw MQTT Messages from MongoDB

MongoDB stores the raw JSON-like messages received from MQTT.

	_id	mysql_reading_id	sensor_id	location	temperature	humidity	air_quality	timestamp	status	alert
480	6a385b3a39cb69b7f001be5e	2	S002	Room B	33.51	80.86	132	2026-06-21 23:44:22	DANGER	{"alert_type":"H
479	6a385b3a39cb69b7f001be5f	3	S003	Laboratory	28.6	52.93	57	2026-06-21 23:44:24	NORMAL	None
478	6a385b3a39cb69b7f001be60	4	S003	Laboratory	20.91	69.29	156	2026-06-21 23:44:26	DANGER	{"alert_type":"F
477	6a385b3c39cb69b7f001be61	5	S002	Room B	27.84	50.11	100	2026-06-21 23:44:28	NORMAL	None
476	6a385b3e39cb69b7f001be62	6	S001	Room A	35.3	52.9	130	2026-06-21 23:44:30	DANGER	{"alert_type":"H
475	6a385b4039cb69b7f001be63	7	S002	Room B	23.59	54.34	36	2026-06-21 23:44:32	NORMAL	None
474	6a385b4239cb69b7f001be64	8	S003	Laboratory	21.51	63.86	161	2026-06-21 23:44:34	DANGER	{"alert_type":"F
473	6a385b4439cb69b7f001be65	9	S002	Room B	29.6	66.97	168	2026-06-21 23:44:36	DANGER	{"alert_type":"F
472	6a385b4639cb69b7f001be66	10	S003	Laboratory	18.24	68.74	96	2026-06-21 23:44:38	NORMAL	None
471	6a385b4839cb69b7f001be67	11	S001	Room A	23.77	77.07	54	2026-06-21 23:44:40	WARNING	{"alert_type":"E

Figure E14: Streamlit dashboard displaying raw MQTT messages stored in MongoDB.

```

    }
  }
])

```

```

env_monitoring> db.sensor_messages.find().pretty()
[
  {
    _id: ObjectId('6a385b3439cb69b7f001be5d'),
    mysql_reading_id: 1,
    sensor_id: 'S002',
    location: 'Room B',
    temperature: 30.24,
    humidity: 66.13,
    air_quality: 139,
    timestamp: '2026-06-21 23:44:20',
    status: 'WARNING',
    alert: { alert_type: 'ENVIRONMENT_WARNING', message: 'One or more environmental values are above the normal range.' },
    stored_at: '2026-06-21 23:44:20'
  },
  { mysql_reading_id: 2, sensor_id: 'S002', location: 'Room B', temperature: 33.51, humidity: 80.86, air_quality: 132, status: 'DANGER' }
]

```

Figure 7: MongoDB documents and aggregation result

9.6 Neo4j Graph Test

Neo4j was tested using the Neo4j browser. The first Cypher query displayed the relationship between sensors and readings:

```

MATCH (s:Sensor)-[:PRODUCED]->(r:Reading)
RETURN s, r
LIMIT 25;

```

A second Cypher query showed reading-alert relationships:

```

MATCH (r:Reading)-[:TRIGGERED]->(a:Alert)
RETURN r, a
LIMIT 25;

```

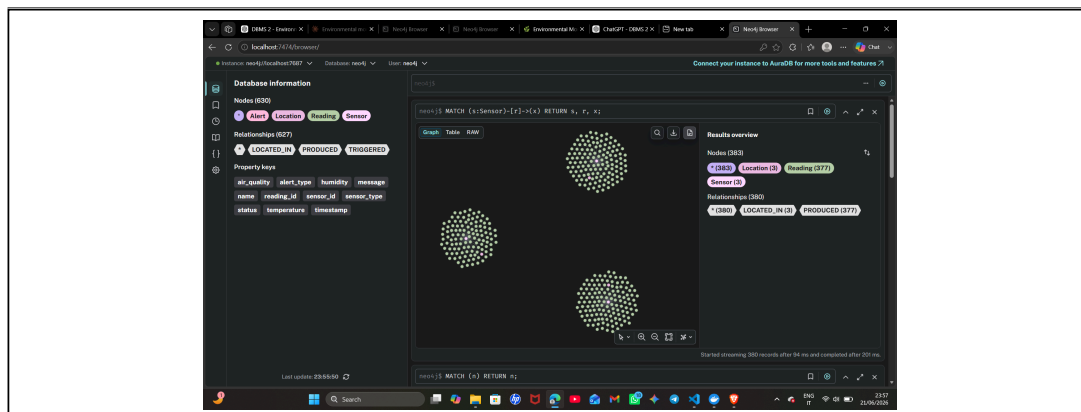


Figure 8: Neo4j graph showing sensor, reading, and alert relationships

Neo4j Graph Verification

Neo4j was used to represent sensors, locations, readings, and alerts as graph nodes connected by relationships. These screenshots are placed directly after the Neo4j graph-test explanation because they show the actual graph database in Neo4j Browser.

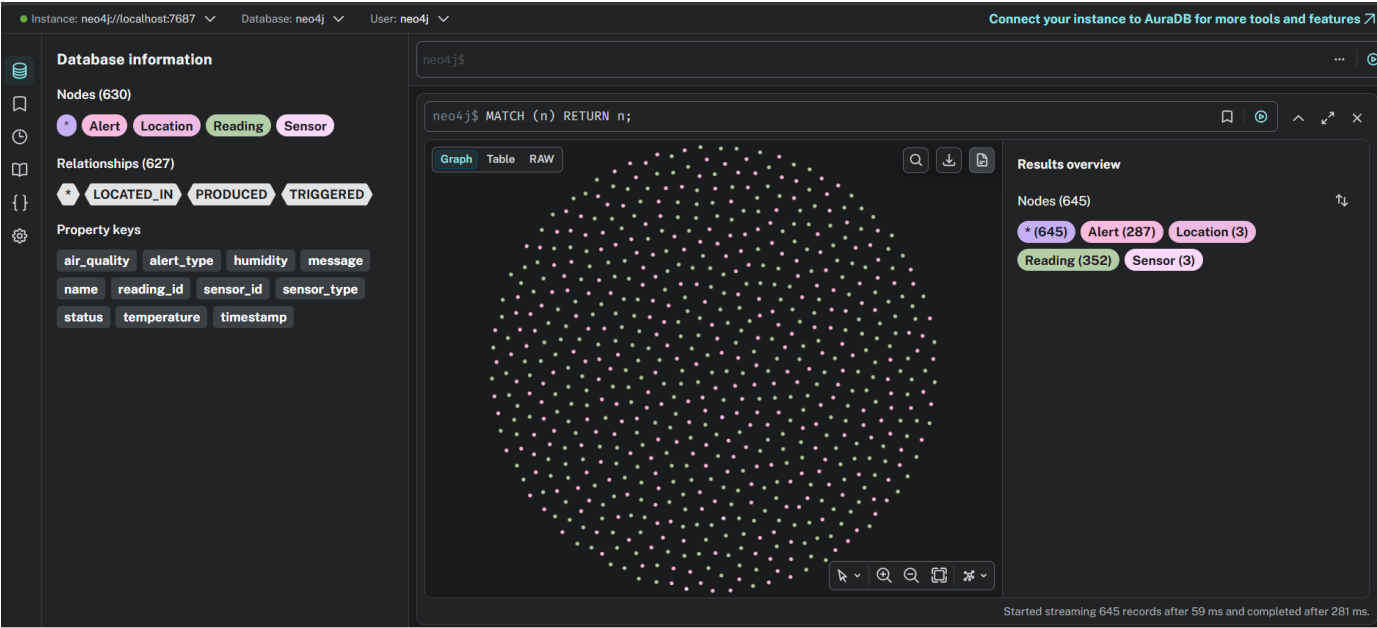


Figure E15: Neo4j Browser showing the full graph overview with Sensor, Location, Reading, and Alert nodes.

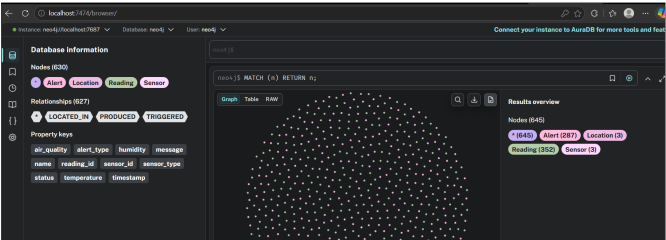


Figure E16: Neo4j database information showing node labels and relationship types.

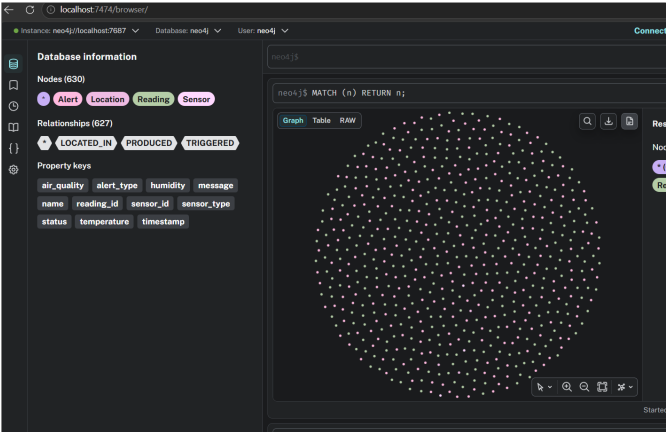


Figure E17: Zoomed Neo4j graph visualization returned by MATCH (n) RETURN n.

Neo4j Relationship Verification

The graph queries also verify the relationship model. Sensors are connected to produced readings and to their physical locations. This supports the graph model described in the report.

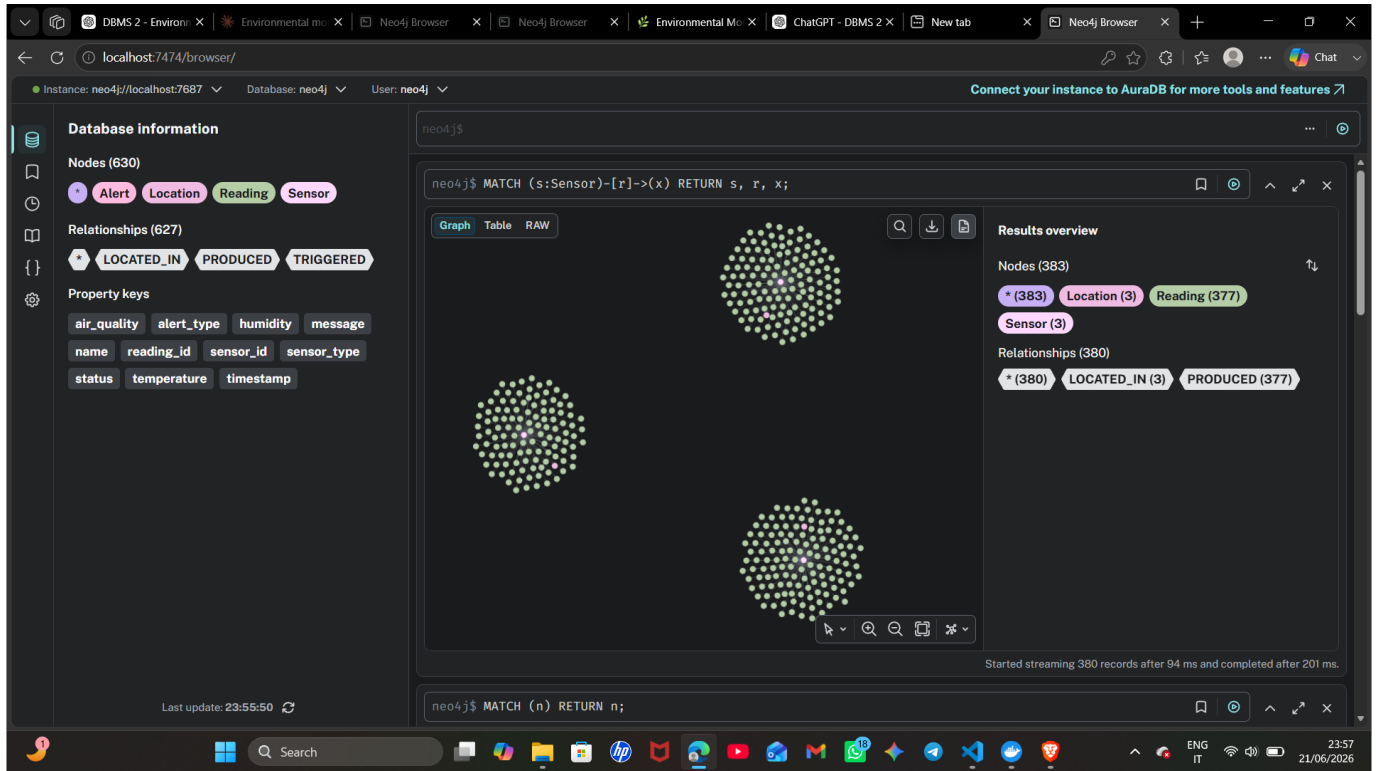


Figure E18: Neo4j graph query showing Sensor nodes connected to related graph nodes through relationships.

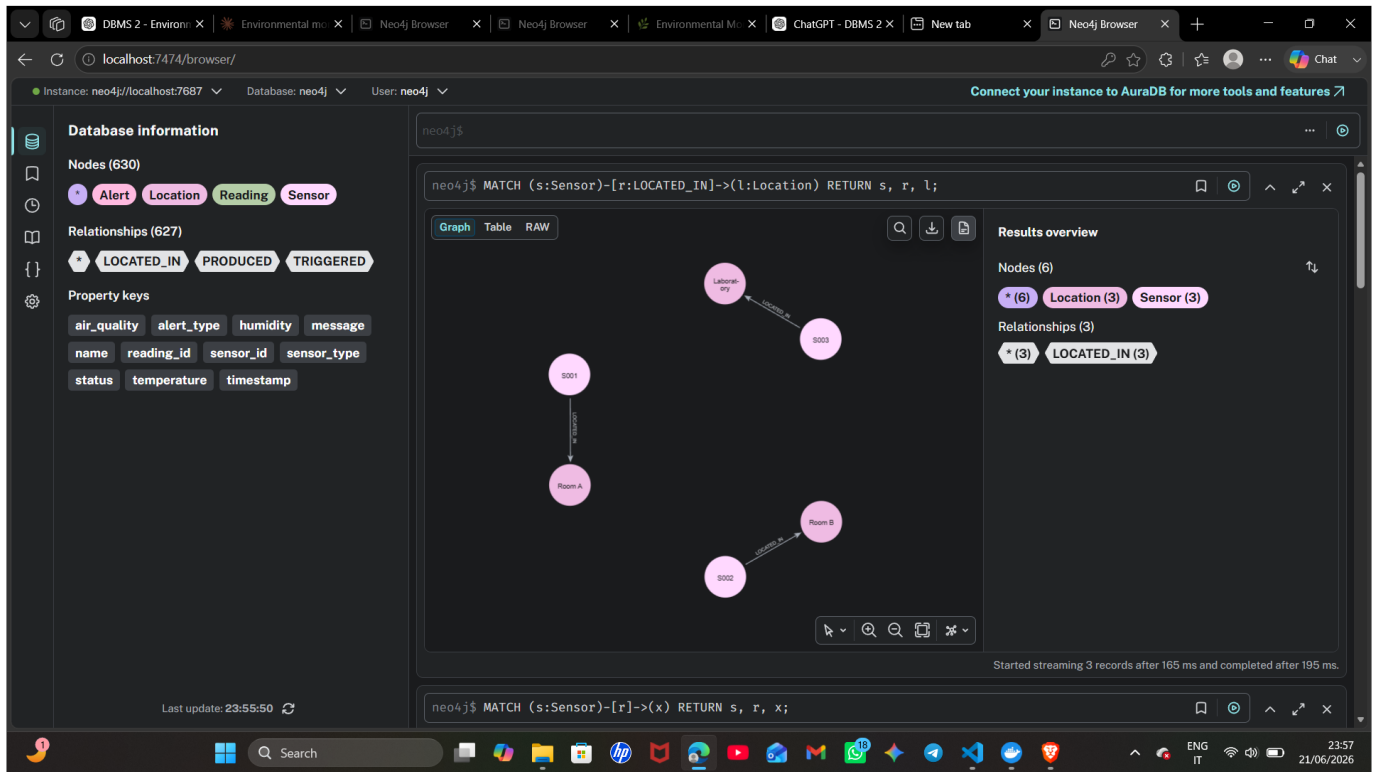


Figure E19: Neo4j LOCATED_IN relationships connecting S001, S002, and S003 to Room A, Room B, and Laboratory.

9.7 Testing Summary

The tests confirmed that the system works correctly from end to end. The publisher generated environmental data, the MQTT broker delivered the messages, the subscriber processed the messages, and the data was stored in MySQL, MongoDB, and Neo4j.

The testing results show that Docker successfully ran all required services, the Python publisher generated sensor data, the MQTT broker delivered the messages, the Python subscriber received and processed the data, MySQL stored structured readings and alerts, MongoDB stored MQTT messages as documents, and Neo4j represented the data as connected graph nodes.

10 Results Discussion

The final system worked successfully as an end-to-end IoT data pipeline. The Python publisher simulated environmental sensors and generated readings for temperature, humidity, and air quality. These readings were sent as JSON messages through the MQTT broker. The Python subscriber received the messages, processed the values, classified each reading, and stored the results in three different databases.

One important result of the project is that the same sensor data was represented in different database models. MySQL stored the data in a structured relational format using tables, primary keys, and foreign keys. MongoDB stored the messages as flexible JSON-like documents. Neo4j stored the data as a graph, showing relationships between sensors, locations, readings, and alerts.

This showed that different database systems are useful for different purposes. MySQL was useful for structured queries and reliable relational storage. MongoDB was useful for storing the original MQTT-style messages. Neo4j was useful for visualizing relationships and understanding how sensors, readings, and alerts are connected.

The testing screenshots confirmed that the system was able to publish data, receive data, process data, store data, and query the stored information. This means the main objective of the project was achieved.

11 Challenges Faced

During the project, some technical challenges appeared. The first challenge was setting up the required services. Since the project needed an MQTT broker, MySQL, MongoDB, and Neo4j, installing each service manually would have been more difficult and time-consuming. To solve this, Docker Compose was used to run all services in containers.

Another challenge was connecting Python to different databases at the same time. Each database required a different Python library and a different connection method. MySQL used `mysql-connector-python`, MongoDB used `pymongo`, and Neo4j used the `neo4j` Python driver. This required careful configuration in the `.env` file.

Streamlit Dashboard Verification

A Streamlit dashboard was used to present the collected data from the three databases. It verifies that the system can display relational data, document data, and graph summaries in one interface.

```
(.venv) PS C:\Users\adona...\environmental_monitoring_mqtt> streamlit run dashboard\app.py
2026-06-21 23:59:26.501  Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL:   http://localhost:8501
Network URL: http://10.2.0.81:8501
```

Figure E20: Running the Streamlit dashboard from Visual Studio Code using streamlit run dashboard\app.py.

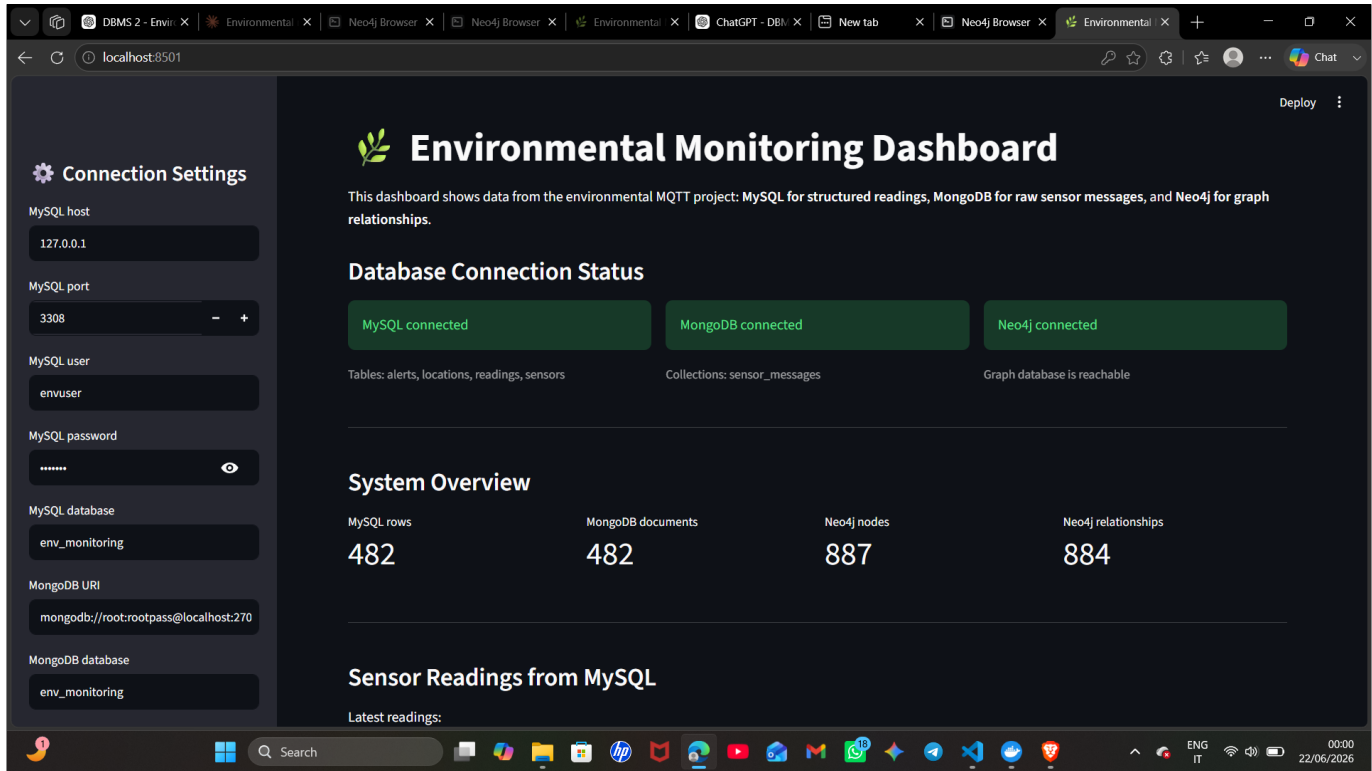


Figure E21: Dashboard connection status confirming MySQL, MongoDB, and Neo4j connections.

Dashboard Data Views

The dashboard displays recent MySQL readings, trends over time, raw MongoDB messages, and a Neo4j graph summary. These views confirm the end-to-end result of the project.



Figure E22: Dashboard showing recent MySQL readings and environmental values over time.

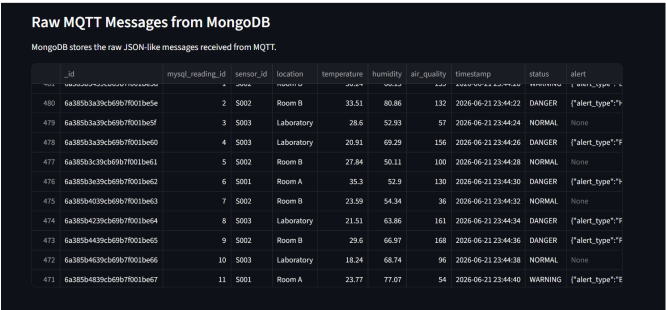


Figure E23: Dashboard view of MongoDB MQTT messages.

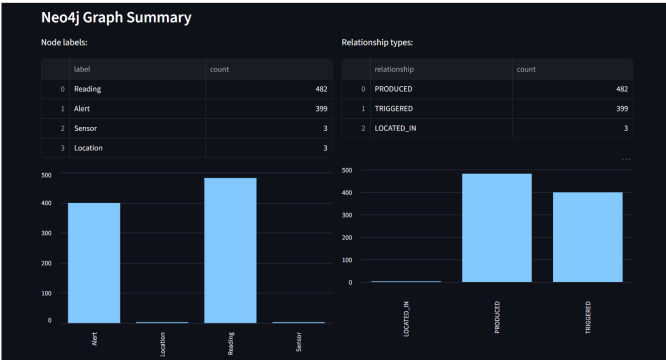


Figure E24: Dashboard summary of Neo4j node labels and relationship types.

A third challenge was making sure the data was stored consistently across the three databases. For example, when a reading was inserted into MySQL, the generated `reading_id` was also used in MongoDB and Neo4j. This helped connect the same reading across the different storage systems.

These challenges helped improve the project because they made the implementation more realistic and closer to how real IoT database systems work.

12 Conclusion

This project implemented an IoT-based environmental monitoring system using MQTT, Python, MySQL, MongoDB, and Neo4j. The system simulated environmental sensors that generated temperature, humidity, and air-quality readings. The readings were sent through an MQTT broker, received by a Python subscriber, processed, classified, and stored in multiple database systems.

The project demonstrated the role of DBMS technologies in IoT applications. MySQL was used to store structured relational data, MongoDB was used to store flexible document data, and Neo4j was used to represent graph relationships. This made the project useful from a database point of view because it showed how the same data can be modeled and queried in different ways.

The final result proved that the system works correctly. The publisher successfully sent data, the subscriber successfully received and processed it, and the databases successfully stored and analyzed the collected readings.

Overall, the project achieved its objective of building a working environmental monitoring system with MQTT communication and multi-database storage.

13 Future Improvements

Although the project works correctly, it can be improved in several ways.

First, real physical sensors could be added instead of simulated data. For example, a real temperature and humidity sensor connected to an Arduino, ESP32, or Raspberry Pi could send actual environmental readings to the MQTT broker.

Second, a web dashboard could be developed to display the readings in real time. The dashboard could show charts for temperature, humidity, and air quality, and it could highlight dangerous values.

Third, user authentication could be added so that only authorized users can access the monitoring system.

Fourth, the alert system could be improved by sending email or mobile notifications when dangerous environmental values are detected.

Finally, more advanced analysis could be added. For example, the system could calculate daily averages, detect unusual patterns, or predict future environmental conditions

based on previous readings.

These improvements would make the system more practical and closer to a real industrial or smart-building monitoring application.

14 References

References

- [1] Docker Documentation. Docker Compose documentation. Available at: <https://docs.docker.com/compose/>
- [2] Eclipse Mosquitto Documentation. MQTT broker documentation. Available at: <https://mosquitto.org/documentation/>
- [3] Eclipse Paho Documentation. Python MQTT client documentation. Available at: <https://www.eclipse.org/paho/>
- [4] MySQL 8.0 Documentation. Relational database documentation. Available at: <https://dev.mysql.com/doc/>
- [5] MongoDB Documentation. Document database documentation. Available at: <https://www.mongodb.com/docs/>
- [6] Neo4j Documentation. Graph database and Cypher documentation. Available at: <https://neo4j.com/docs/>
- [7] Python Documentation. Python programming language documentation. Available at: <https://docs.python.org/3/>